

Objetos

Conceito

O conceito de objetos deve ser entendido como o formato disponibilizado na linguagem para **estruturar informações**. Ou seja, para estruturar o número do **pi** em um objeto python é necessário usar a estrutura (objeto) **float**. Para estruturar a **coleção** de probabilidades de ocorrências dos dígitos de 1 a 9 é possível usar a estrutura **dictionary**. Para estruturar um **nome**, recomenda-se o uso de uma **string**. O contexto da aplicação e a experiência do programador são fundamentais na **escolha** da estrutura python **adequada** para a representação de informações desestruturadas "pythonicamente".

Principais tipos de objetos existentes nativamente

- int Representação de números **inteiros** negativos e positivos
- float Representação de números **reais** negativos e positivos
- string Representação de uma **coleção de caracteres**
- list Representação de uma **coleção de elementos** mutável
- tuple Representação de uma **coleção de elementos** imutável
- dict Representação de uma **coleção de elementos** pareados
- bool Representação de uma informação **binária**

Sintaxe Python (int)

EXAMPLE

2

EXAMPLE

10

EXAMPLE

321

Observação

Os objetos
int
podem ou não
ser armazenadas em variáveis

Objetos

Sintaxe Python (float)

Observe

O
ponto



EXAMPLE

1.6180

EXAMPLE

2.7172

EXAMPLE

3.1415

Observação
Os objetos
float
podem ou não
ser armazenados em variáveis

Sintaxe Python (string)

Observe

as
aspas



EXAMPLE

“qualquer texto”

EXAMPLE

“prandiano”

EXAMPLE

“3.1415” → string

Observação
Os objetos
string
podem ou não
ser armazenados em variáveis

As strings são **coleções** de caracteres. Sendo uma coleção, esse objeto possui a **sintaxe de fatiamento** (slicing) como segue:

	0	1	2	3	4	5	6	7	8	índices positivos
nome_escola =	p	r	a	n	d	i	a	n	o	
	-9	-8	-7	-6	-5	-4	-3	-2	-1	índices negativos

retorna o **primeiro** elemento da coleção

nome_escola[0] → “p”

retorna o **quarto** elemento da coleção

nome_escola[3] → “n”

retorna os elementos de índice **3 até** o índice **5**

nome_escola[3:6] → “ndi”

retorna o **último** elemento da coleção

nome_escola[-1] → “o”

retorna os elementos de índice **-4 até** o índice **-2**

nome_escola[-4:-1] → “ian”

leitura humana

sintaxe python

retorna

Curso de Python

Slicing
1 / 2

Fatiamento
1 / 2

Objetos

Slicing

2 / 2

retorna os elementos de índice **0** até o índice **2**
um elemento por vez

retorna os elementos de índice **1** até o índice **7**
dois elementos por vez

retorna os elementos de índice **-8** até o índice **-1**
dois elementos por vez

retorna os elementos de índice **-1** até o índice **-8**
dois elementos por vez na ordem inversa

retorna os elementos do começo ao fim
um elemento por vez na ordem inversa

leitura humana

nome_escola[0:3:1] → "pra"

nome_escola[1:8:2] → "rnin"

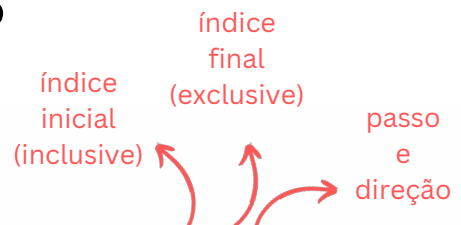
nome_escola[-8:-1:2] → "rnin"

nome_escola[-1:-8:-2] → "ninr"

nome_escola[::-1] → "onaidnarp"

sintaxe python

retorna



Observações

1. Fatiamento de **string** retorna uma **string**!!!
2. Todos os parâmetros do fatiamento (índice inicial, índice final e passo) são **opcionais**, ou seja, se não forem fornecidos, possuem um valor **padrão**. Se o parâmetro inicial não for fornecido, o fatiamento ocorre **desde o primeiro elemento**. Se o parâmetro índice final não for fornecido, o fatiamento ocorre **até o último elemento**. E, finalmente, se o passo não for fornecido, o fatiamento ocorre **um elemento por vez na direção normal**.

Sintaxe Python (list)

Observe
OS
colchetes



EXAMPLE ["qualquer objeto",]

EXAMPLE [1, 2, 3, 4, 5, 6]

EXAMPLE [1, 1.61, 2, 3]

um elemento
dentro da lista

seis elementos
dentro da lista

quatro elementos
dentro da lista

Observação
Os objetos
list
podem ou não
ser armazenadas em variáveis

Resumo

As listas são coleções de qualquer objeto. Sendo uma coleção, esse objeto possui a **sintaxe de fatiamento** (slicing). Todas as regras de fatiamento das strings servem igualmente às listas. Porém, o fatiamento de uma lista retorna uma lista!!!

Os elementos devem ser separados por vírgula.

Curso de Python

Objetos

Sintaxe Python (tuple)

Observe
OS
parêntesis



EXAMPLE

("qualquer objeto",)

um elemento
dentro da **tupla**

EXAMPLE

(1, 2, 3, 4, 5, 6)

seis elementos
dentro da **tupla**

EXAMPLE

(1, 1.61, 2, 3)

quatro elementos
dentro da **tupla**

Observação

Os objetos
tuple
podem ou não
ser armazenadas em variáveis

Resumo

As tuplas são coleções de qualquer objeto. Sendo uma coleção, esse objeto possui a **sintaxe de fatiamento** (slicing). Todas as regras de fatiamento das strings servem igualmente às tuplas. Porém, o fatiamento de uma tupla retorna uma tupla!!! Os elementos devem ser separados por vírgula e blocados por parêntesis.

Observações

1. O principal fator que diferencia as listas das tuplas, é que a lista é um objeto **mutável**, ou seja, sempre é possível **acrescentar, remover, modificar** elementos da lista. Porém, as tuplas **nunca** permitem essas operações; as tuplas são **imutáveis**;
2. Para se criar uma tupla com apenas um elemento (quando necessário for), é necessário incluir uma vírgula no final ("qualquer objeto",). A vírgula incluída no final, explicita que se trata de uma coleção. Como a coleção é limitada por parêntesis, a coleção será do tipo tupla.

Sintaxe Python (dictionary)

Observe
as
chaves e os dois
pontos



EXAMPLE

{1: 0.301}

um elemento (par)
dentro do **dicionário**

EXAMPLE

{1: "um", 2: "dois"}

chave
1

valor
1

chave
2

valor
2

Observação

Os objetos
dictionary
podem ou não
ser armazenadas em variáveis

Objetos

Resumo

Os dicionários são coleções de pares chave:valor separados por vírgula. Sendo uma coleção, esse objeto possui a **sintaxe de fatiamento** (slicing). As regras de fatiamento dos dicionários são específicas para este objeto.

```
dicionario = {1: "int", 1.0: "float", "1": "string"}
```

Fatiamento

retorna o **valor** associado com a chave **1**

retorna o **valor** associado com a chave **1.0**

retorna o **valor** associado com a chave **"1"**

leitura humana

nome_escola[1] → "int"

nome_escola[1.0] → "float"

nome_escola["1"] → "string"

sintaxe python

retorna

Slicing

Observações

1. Os dicionários são fatiados pelas suas chaves, que por sua vez, estão associadas com seus valores;
2. Não existe índices (posições) nos dicionários;
3. Os dicionários são objetos não ordenados;
4. Os dicionários são unidirecionais, ou seja, dada uma chave, é possível obter o valor associado. Nunca o contrário;
5. Essa estrutura de mapeamento chave:valor permite uma execução computacional de procura muito mais rápida do que nas demais coleções. Para objetos com muitos elementos (pares) essa performance melhorada dos dicionários é mais percebida.

Objetos

Sintaxe Python (boolean)

Observe
as
letras 'T' e 'F'
Maiúsculas



EXAMPLE

EXAMPLE

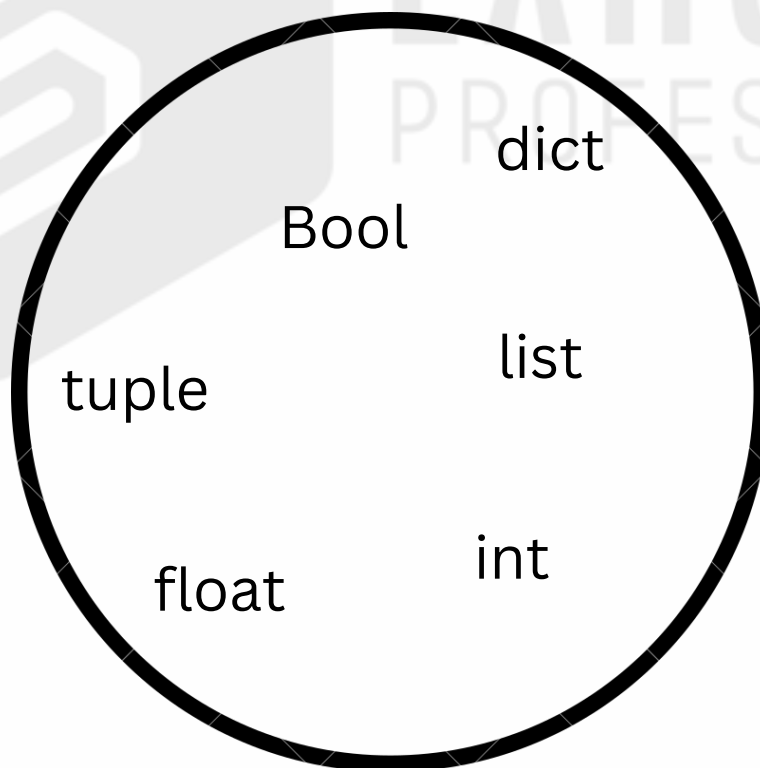
False

True



Resumo

Os objetos booleanos permitem operações de álgebra de boole (and, or, not, xor). As informações que podem ser representadas por booleanos deve ser do tipo binárias (por exemplo, ligado-desligado, aceso-apagado, horizontal-vertical, etc.).



Objetos Nativos do tipo Built-in

Curso de Python